Ansible Playbooks & Windows

Arseniy Buzyan

Ansible för Windows – Varför och Hur?

- Ansible är inte bara för Linux – det kan hantera Windows-servrar lika enkelt!
- Skillnad: Linux använder SSH, Windows använder WinRM (Windows Remote Management) – en inbyggd SOAP-baserad protokoll för fjärrhantering.

Arkitekturen bakom plattformsoberoende automation



Enligt Red Hat: Ansible Automation Platform har fullt testade Windows-moduler för att hantera allt från IIS till Active Directory.

Varför Ansible för Windows?

Problem	Lösning med Ansible
Manuell konfiguration av IIS, DNS, Hyper-V	Automatisk installation med win_feature-modulen
Olika servrar blir olika över tid	Samma playbook = samma resultat varje gång
Svårt att hantera 100+ Windows-servrar	Ett kommando – alla servrar uppdateras
Inga dokumenterade processer	Playbooks är självdokumenterande kod

Beprövad automation i **storskalig** produktion

6,5h → 20 min

Tidsåtgång för patching per Windows-server.

40 000

Frigjorda timmar från manuellt arbete.

4 000

Automatiserade ärenden via ServiceNow-integration.

ABB

Sparar 57 timmar per månad/team på automatiserad patching.

Volvo

Standardiserar serverkonfiguration via Playbooks.

Skatteverket

Experimenterar med AI-drivna playbooks för Windows.

Windows-moduler – Ditt Verktygsbälte

Modul	Vad gör den?	Användning
win_ping	Testa anslutning till Windows-värd	ansible windows -m win_ping
win_feature	Installera/ta bort Windows-funktioner (IIS, DNS, Hyper-V)	win_feature name=Web-Server state=present
win_service	Hantera Windows-tjänster	win_service name=W3SVC state=started
win_copy	Kopiera filer till Windows	win_copy src=index.html dest=C:\inetpub\wwwroot\
win_file	Hantera filer och mappar	win_file path=C:\temp state=directory
win_shell	Kör PowerShell-kommandon	win_shell Get-Service
win_firewall	Hantera brandväggsregler	win_firewall
win_updates	Installera Windows-uppdateringar	win_updates

Ansible har dedikerade Windows-moduler – alla börjar med win_.

Enligt Ansible-dokumentation: Dessa moduler är skrivna i PowerShell och fungerar bara på Windows-värdar . Ansible använder WinRM för att ansluta.

Tre steg till fullständig Windows-kontroll

1. Windows (Target)

```
Enable-PSRemoting -Force
Set-Service -Name WinRM
-StartupType 'Automatic'
New-NetFirewallRule -Name
'WinRM-HTTP' -DisplayName
'WinRM HTTP' -Enabled True
-Direction Inbound -Protocol
TCP -LocalPort 5985
```

2. Kontrollnoden (WSL2)

```
pip install pywinrm
```

3. Test (Verifiering)

```
ansible windows -i
inventory.ini -m win_ping

{
  "changed": false,
  "ping": "pong"
}
```

Förbereda Windows för Ansible (WinRM)

Innan Ansible kan hantera Windows måste WinRM aktiveras och konfigureras.

Steg 1 – Aktivera WinRM på Windows (som Administrator i PowerShell)

powershell

```powershell
# Aktivera WinRM och PSRemoting
Enable-PSRemoting -Force

# Sätt WinRM att starta automatiskt
Set-Service -Name WinRM -StartupType 'Automatic'

# Tillåt grundläggande autentisering (endast labb!)
Set-Item -Path WSMan:\localhost\Service\Auth\Basic -Value
  $true

# Lägg till brandväggsregel för WinRM HTTP (port 5985)
New-NetFirewallRule -Name "WinRM-HTTP" -DisplayName "WinRM
  HTTP" -Enabled True -Direction Inbound -Protocol TCP -
  LocalPort 5985 -Action Allow
```

Steg 2 – Installera pywinrm på kontrollnoden (WSL2)

```
bash
pip install pywinrm
```

Steg 3 – Testa anslutning

bash

```
ansible windows -i inventory.ini -m win_ping
```

Förväntat resultat

```
text
win-server | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

Enligt Red Hat: För produktionsmiljöer rekommenderas HTTPS (port 5986) med certifikatvalidering.

Inventory för Windows – Rätt Konfiguration

variabler för Windows-anslutning.

Exempel – `inventory.ini` för Windows:

ini

```ini
# inventory.ini – Windows-servrar
[windows_servers]
win01 ansible_host=192.168.1.100
win02 ansible_host=192.168.1.101

[windows_servers:vars]
ansible_user=Administrator
ansible_password=SecurePassword123!
ansible_connection=winrm
ansible_winrm_transport=basic
ansible_winrm_port=5985
```

Viktiga variabler

Proffstips: Använd ALLTID **ansible-vault** för lösenord i produktion !

Variabel	Värde	Förklaring
ansible_connection	winrm	Använder WinRM istället för SSH
ansible_winrm_transport	basic, ntlm, kerberos	Autentiseringsmetod
ansible_winrm_port	5985 (HTTP) eller 5986 (HTTPS)	WinRM-port
ansible_user	Administrator	Användarnamn

Välj rätt autentiseringsmetod för din miljö

Metod	Användning	Säkerhet	Specifika Krav
Basic	Labb-miljöer	Låg	Inga extra krav.
NTLM	Lokala konton (ej domän)	Medel	HTTPS rekommenderas.
Kerberos	Domän-miljöer (Active Directory)	Hög	`krb5.conf` (Ansibles rekommenderade val).
Certificate	Produktion	Hög	Certifikat på klient och server.

Exempel – NTLM (vanligast)

ini

```
ansible_winrm_transport=ntlm
ansible_winrm_port=5986
ansible_winrm_scheme=https
ansible_winrm_server_cert_validation=ignore
```

Exempel – Kerberos (domän)

ini

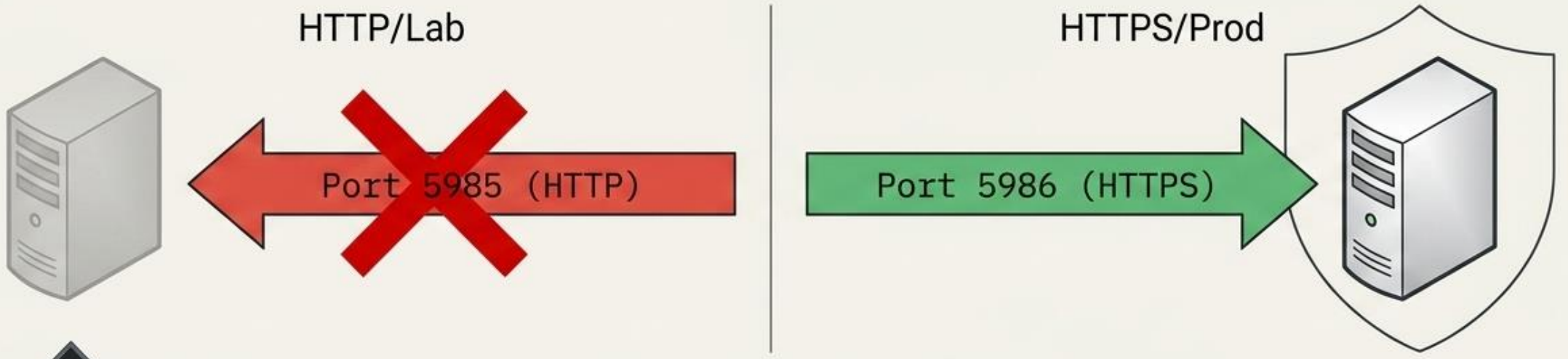
```
ansible_winrm_transport=kerberos
ansible_user=admin@EXAMPLE.COM
```

Enligt Ansible-dokumentation: Kerberos är det rekommenderade valet i Active Directory-miljöer eftersom det ger bäst säkerhet och stöd för enkel inloggning .

HTTPS för WinRM – Produktionsrekommendation

I produktion bör du använda HTTPS (port 5986) istället för HTTP .

Säkra uppkopplingen för produktionsmiljöer




```
Inventory Vars:
ansible_port=5986
ansible_winrm_scheme=https
```

Certifikat: Ändra 'validate=ignore' (självsignerat) till 'validate' (CA-certifikat) för produktion.

Säkerhetstips: Använd ALLTID ansible-vault för hantering av lösenord i produktionsmiljöer!

Gemini Notebook

Steg 1 – Skapa HTTPS-listener på Windows

```powershell
# Skapa självsignerat certifikat (labb) eller använd CA-certifikat
$cert = New-SelfSignedCertificate -DnsName $env:COMPUTERNAME -CertStoreLocation Cert:\LocalMachine\My

# Skapa HTTPS-listener
New-Item -Path WSMan:\localhost\Listener -Transport HTTPS -Address * -CertificateThumbPrint $cert.Thumbprint -Force
```

Steg 2 – Konfigurera säkerhetsinställningar

```powershell
# Tillåt endast HTTPS
Set-Item -Path WSMan:\localhost\Service\AllowUnencrypted -Value $false

# Tillåt grundläggande autentisering (vid behov)
Set-Item -Path WSMan:\localhost\Service\Auth\Basic -Value $true
```

Steg 3 – Använd HTTPS i inventory

ini

```
[windows:vars]
ansible_port=5986
ansible_winrm_scheme=https
ansible_winrm_server_cert_validation=ignore  # För självsignerade certifikat
```

VIKTIGT: För produktionsmiljöer: använd CA-signerade certifikat och sätt ansible_winrm_server_cert_validation=validate.

Windows-moduler – ditt nya verktygsbälte

The Toolkit

Kärnfunktioner	System & Säkerhet
`win_feature` (Installera/ta bort roller som IIS, DNS) `win_service` (Hantera tjänster)	`win_updates` (Installera Windows-uppdateringar) `win_firewall` (Hantera regler)
Filhantering	**Exekvering & Nätverk**
`win_copy` (Kopiera filer från kontrollnod) `win_file` (Hantera mappar/rättigheter)	`win_shell` (Kör rena PowerShell-kommandon) `win_ping` (Testa anslutning)

Alla moduler är skrivna i PowerShell, körs direkt på Windows-värden och kräver WinRM

win_feature – Installera Windows-funktioner

win_feature – Installera Windows-funktioner

win_feature installerar Windows-roller och -funktioner (t.ex. IIS, DNS, Hyper-V) .

Modul-djupdykning: Installation med win_feature

```
- name: Installera IIS
  ansible.windows.win_feature:
    name: Web-Server
    state: present
    include_management_tools: yes
    include_sub_features: yes
```

Definierar önskat tillstånd – installera om det saknas.

Inkluderar GUI-verktyg för administration.

⚠ Installerar ALLT under en funktion. Proffstips: Använd med försiktighet då det onödigt ökar serverns attackyta.

Grundläggande användning

yaml

```yaml
- name: Installera IIS
  ansible.windows.win_feature:
    name: Web-Server
    state: present
```

Installera flera funktioner samtidigt

yaml

```yaml
- name: Installera IIS med vanliga funktioner
  ansible.windows.win_feature:
    name:
      - Web-Server
      - Web-Asp-Net45
      - Web-Mgmt-Console
      - Web-Dyn-Compression
    state: present
```

Viktiga parametrar

Proffstips: Använd include_sub_features: yes med försiktighet – det installerar ALLT under en funktion, vilket ökar attackytan.

Parameter	Vad gör den?	Exempel
name	Namn på funktionen/rollen	Web-Server
state	present = installera, absent = ta bort	state: present
include_sub_features	Installera alla underfunktioner	include_sub_features: yes
include_management_tools	Installera hanteringsverktyg	include_management_tools: yes

win_service – Hantera Windows-tjänster

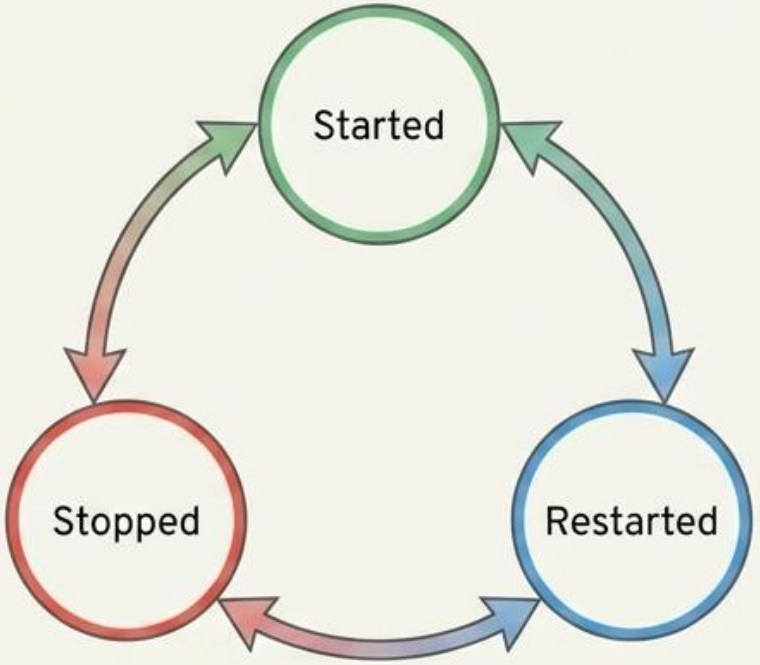
- win_service – Hantera Windows-tjänster
- win_service startar, stoppar och konfigurerar Windows-tjänster .

Grundläggande användning

yaml

```yaml
- name: Starta IIS och sätt automatisk start
  ansible.windows.win_service:
    name: W3SVC
    state: started
    start_mode: auto
```

Modul-djupdykning: Tillståndsstyrd tjänstehantering (win_service)



"state: restarted" -> Kritiskt! Använd detta (inte 'started') när konfigurationer har ändrats för att tvinga fram en omstart av en redan igångvarande tjänst.

state: started	Startar tjänsten.
state: stopped	Stoppar tjänsten.
start_mode: auto	Säkerställer överlevnad efter server-omstart.

Jämförelse av tillstånd

Proffstips: Använd `restarted` istället för `started` när du har ändrat konfiguration – `restarted` garanterar att tjänsten startas om även om den redan är igång.

Parameter	Värde	Effekt
state	started	Startar tjänsten
state	stopped	Stoppar tjänsten
state	restarted	Startar om tjänsten
start_mode	auto	Startar automatiskt vid uppstart
start_mode	manual	Måste startas manuellt
start_mode	disabled	Inaktiverad – kan inte startas

Handlers – Starta Om Efter Ändring

Handlers körs ENDAST om de notifieras – perfekt för att starta om tjänster efter konfigurationsändringar.

Exempel – handler som startar om IIS

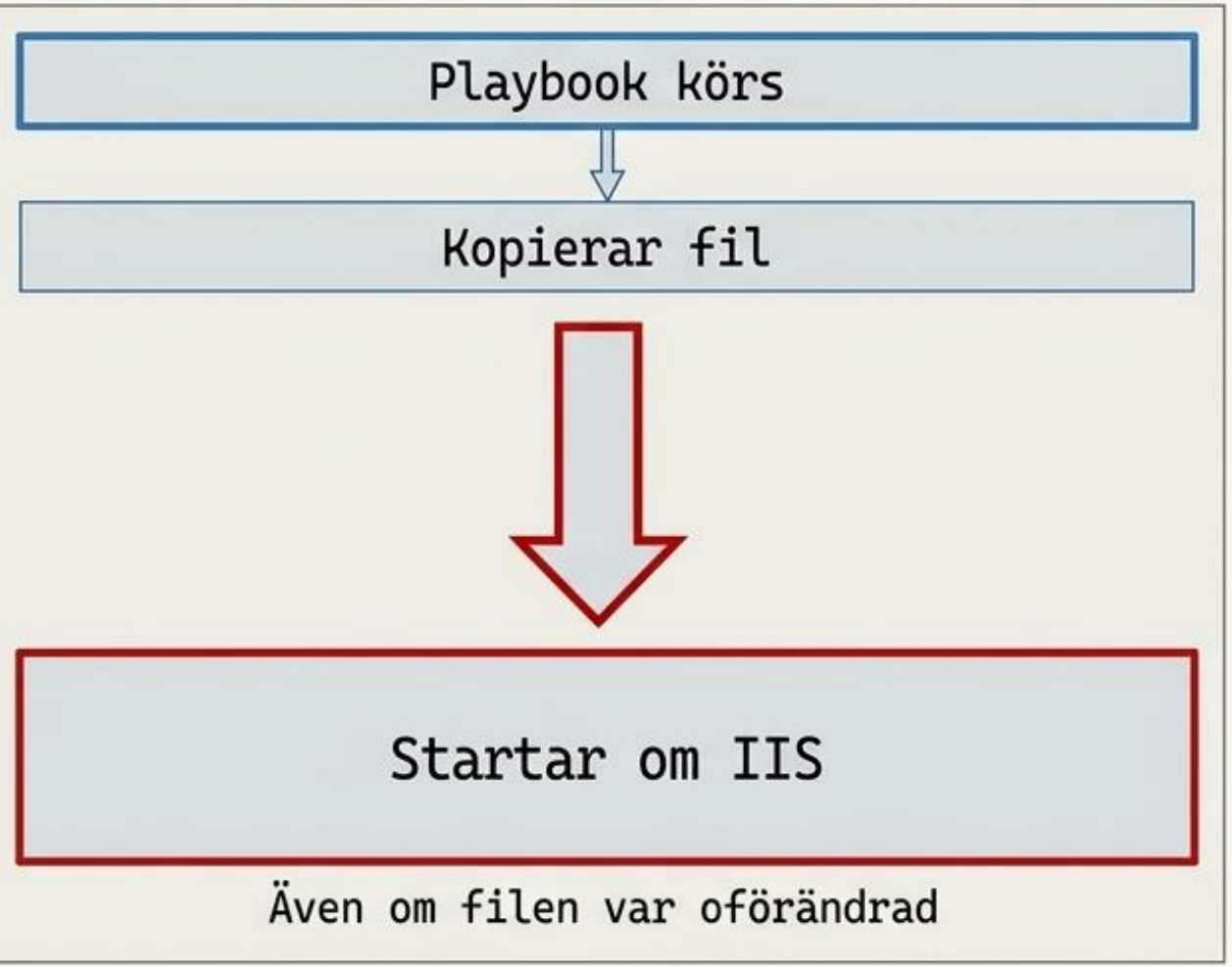
yaml

```yaml
---
- name: Konfigurera IIS-webbplats
  hosts: windows_servers
  tasks:
    - name: Kopiera ny index.html
      ansible.windows.win_copy:
        src: files/index.html
        dest: C:\inetpub\wwwroot\index.html
      notify: Starta om IIS   # Notifierar handlern!

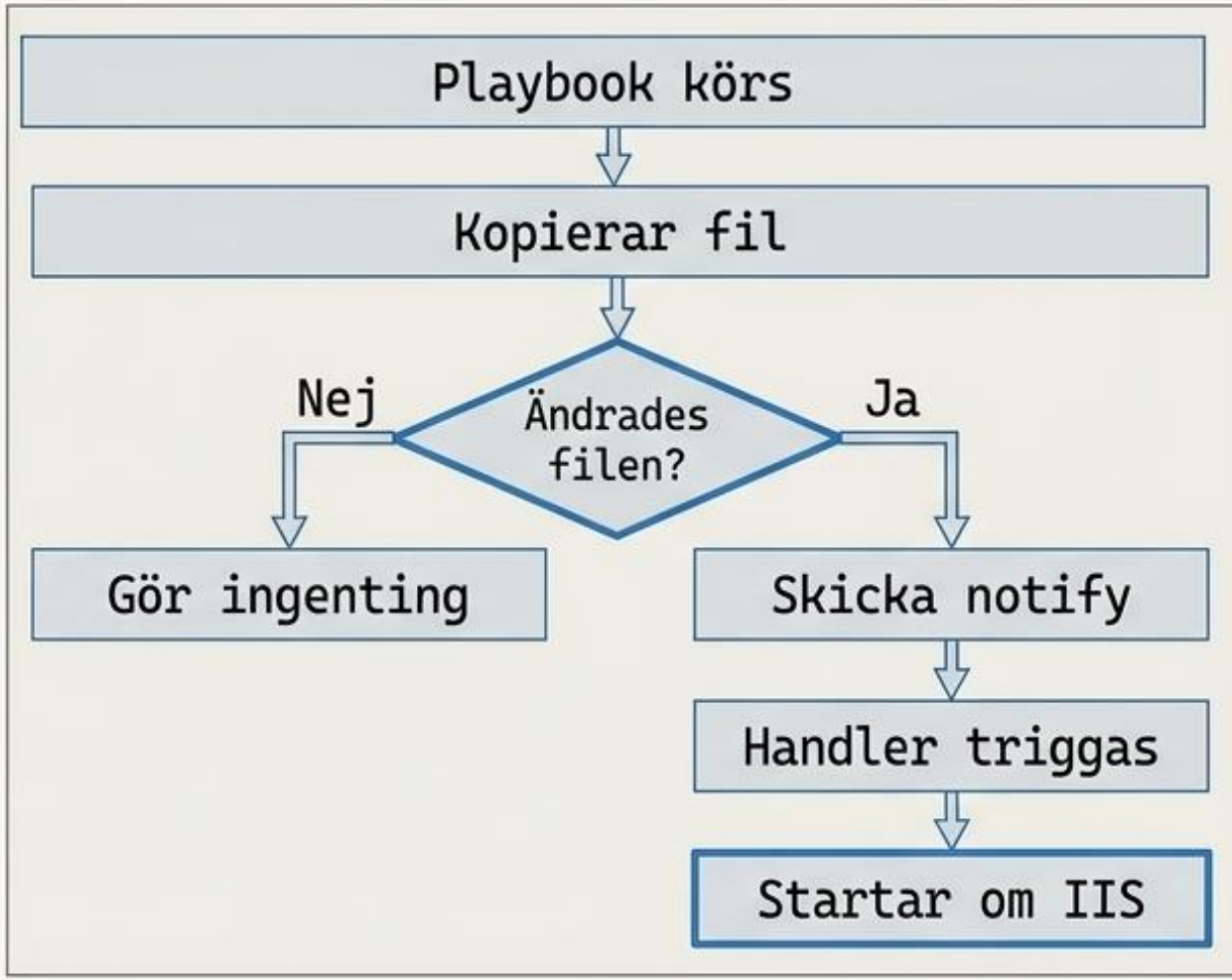
  handlers:
    - name: Starta om IIS
      ansible.windows.win_service:
        name: W3SVC
        state: restarted
```

Idempotens i praktiken med Handlers

Utan handler – Ineffektivt



Med handler – Idempotent



Handlers körs ENDAST om de notifieras om en faktisk ändring.
Detta gör stora automationsmiljöer extremt effektiva.



Mål: Skapa en Ansible-playbook som installerar IIS, startar tjänsten och kopierar en webbsida .

Workshop – Installera IIS med Ansible

Steg 1 – Skapa inventory.ini

ini

```
[windows]
win01 ansible_host=192.168.1.100
  ansible_user=Administrator
  ansible_password=YourPassword!
  ansible_connection=winrm
  ansible_winrm_transport=basic
  ansible_winrm_port=5985
```

Steg 2 – Skapa files/index.html

```html
<h1>Välkommen till Ansible!</h1>
<p>Denna sida skapades automatiskt.</p>
```

inventory.ini

```
[windows]
win01 ansible_host=192.168.1.100

[windows:vars]
ansible_connection=winrm
ansible_winrm_transport=basic
ansible_winrm_port=5985
```

files/index.html

```
<h1>Välkommen till Ansible!</h1>
<p>Denna sida skapades automatiskt med
Infrastructure as Code.</p>
```

Mål: Konfigurera en inventory-fil och förbereda webbsidans innehåll innan exekvering.

Steg 3 – Skapa install_iis.yml

```yaml
---
- name: Installera och konfigurera IIS
  hosts: windows
  tasks:
    - name: Installera IIS
      ansible.windows.win_feature:
        name: Web-Server
        state: present
        include_sub_features: yes
        include_management_tools: yes

    - name: Starta IIS
      ansible.windows.win_service:
        name: W3SVC
        state: started
        start_mode: auto

    - name: Kopiera webbsida
      ansible.windows.win_copy:
```

install_iis.yml

```yaml
- name: Installera IIS
  ansible.windows.win_feature:
    name: Web-Server
    state: present

- name: Starta IIS
  ansible.windows.win_service:
    name: W3SVC
    state: started

- name: Kopiera index.html
  ansible.windows.win_copy:
    src: files/index.html
    dest: C:\inetpub\wwwroot\index.html
```

Task 1: win_feature

Task 2: win_service

Task 3: win_copy

Playbooken läser uppifrån och ner, och garanterar att varje beroende är uppfyllt innan nästa steg körs.

Steg 4 – Kör playbook

bash

```
ansible-playbook -i inventory.ini install_iis.yml
```

Workshop (Fas 3): Exekvering och verifiering

```
$ ansible-playbook -i inventory.ini install_iis.yml
```

http://win01

Välkommen till Ansible!

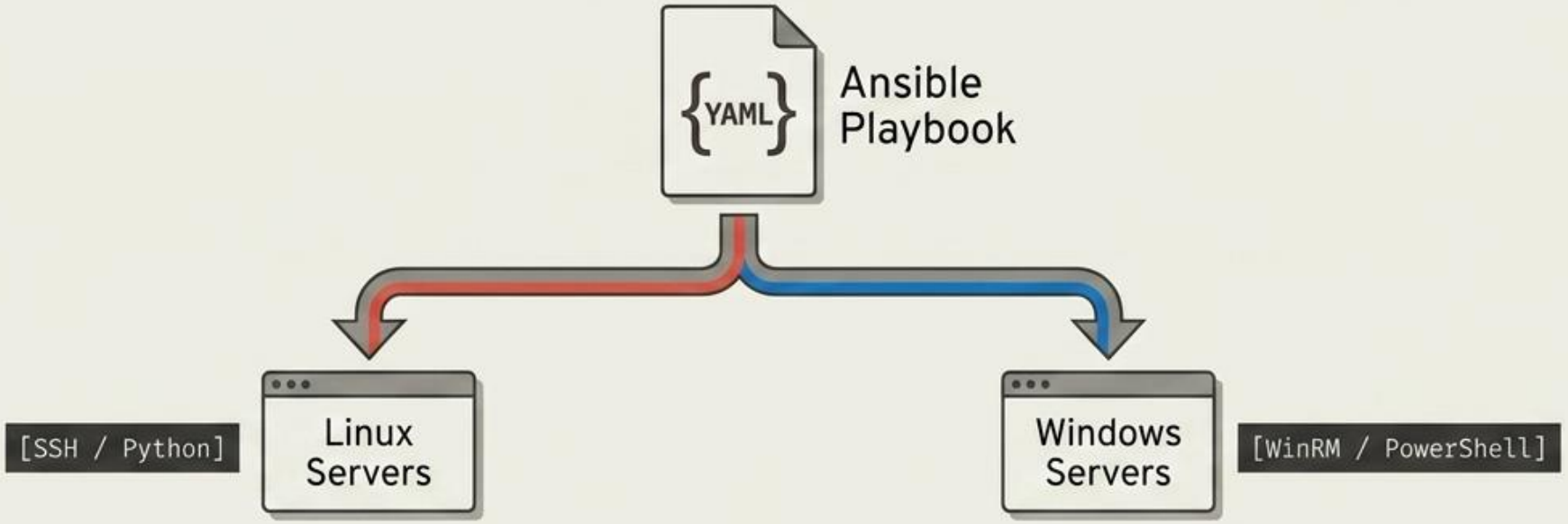
Pro-tip: Modulen `win_feature` kan returnera statusen ‘`reboot_required`’. Detta kan fångas upp i koden för att vid behov automatiskt starta om servern under pågående playbook-körning.

Steg 5 – Verifiera: Öppna `http://win01` i webbläsaren

Enligt Red Hat: `win_feature` returnerar `reboot_required`

du kan använda detta för att starta om servern vid behov .

Ett gemensamt språk för hela infrastrukturen



Genom att använda inbyggda protokoll som **WinRM** överbryggar Ansible klyftan **mellan** operativsystem. Ni behöver inte längre separata **verktygssilor** för Linux och Windows. Samma kod, samma säkerhetskrav, och samma idempotenta resultat över hela er **hybridmiljö. Redo för produktion.**

Sammanfattning

- Windows-moduler: Ansible har dedikerade Windows-moduler (win_ping, win_feature, win_service, win_copy)
- WinRM: Windows använder WinRM istället för SSH – måste aktiveras och konfigureras
- HTTPS: Använd HTTPS (port 5986) i produktion med certifikatvalidering
- Autentisering: Välj rätt metod (Basic, NTLM, Kerberos, Certificate) baserat på miljö
- Idempotens: Playbooks är idempotenta – kör om och om igen utan negativa bieffekter
- Handlers: Notifiera handlers för att starta om tjänster ENDAST vid ändringar

Quiz

https://forms.gle/JhoFXiMdxDzXB7iq5